Абстракция: Процесс

Рассмотрим наиболее фундаментальную абстракцию, которую ОС предоставляет пользователям: процесс. Определение процесса довольно-таки просто - это работающая программа. Программа сама по себе является безжизненной вещью, располагающейся на диске - это набор инструкций и возможно каких-то статических данных, ожидающих момента запуска. Именно ОС берет эти байты и запускает их, преобразую программу во что-то полезное.

Чаще всего пользователи желают запускать более одной программы одновременно, например вы можете запустить на вашем ноутбуке браузер, игру, медиаплеер, текстовый редактор и тому подобное. Фактически типичная система может запускать десятки и сотни процессов одновременно. Этот факт делает систему более простой в использовании, вам никогда не приходится беспокоиться о том что CPU свободен, вы просто запускаете программы.

Отсюда вытекает проблема: как обеспечить иллюзию множества CPU? Как ОС создать иллюзию практически бесконечного количества CPU даже если у вас всего один физический CPU?

ОС создает эту иллюзию посредством виртуализации CPU. Запуская один процесс, затем останавливая его, запуская другой процесс и так далее ОС может поддерживать иллюзию того, что существует множество виртуальных CPU, хотя фактически это будет один или несколько физических процессоров. Такая техника называется разделение ресурсов CPU по времени. Она позволяет пользователям запускать столько одновременных процессов, сколько они пожелают. Ценою такого решения является производительность - поскольку каждый процесс будет обрабатываться медленнее, если CPU делят несколько процессов.

Для воплощения виртуализации CPU, а особенно для того чтобы делать это хорошо, ОС нуждается и в низкоуровневой и в высокоуровневой поддержке. Низкоуровневая поддержка называется механизмами - это низкоуровневые методы или протоколы, которые реализуют нужную часть функционала. Пример такого функционала - контекстное переключение, которое дает ОС возможность останавливать одну программу и запускать на процессоре другую программу. Такое разделение по времени реализовано во всех современных ОС.

На вершине этих механизмов располагается некоторая логика, заложенная в ОС, в форме политик. Политика - некоторый алгоритм принятия решения операционной системой. Такие политики например отвечают на вопрос, какую из списка программ нужно запустить в первую очередь. Так например на этот вопрос ответит политика называющаяся планировщик (scheduling policy) и при выборе решения будет руководствоваться такими данными как история запуска (какая программа была запущена

дольше всех за последнюю минут), какую нагрузку осуществляет данный процесс (????) (какие типы программ были запущены), метрики производительности (оптимизирована ли система для интерактивного взаимодействия или для пропускной способности).

Абстракция: процесс

Абстракция работающей программы, выполняемая операционной системой это то что мы называем процесс. Как уже было сказано ранее процесс - просто работающая программа, в любой моментальный промежуток времени по которой мы можем получить суммарную информацию с различных ресурсов системы к которым обращается или которые эта программа затрагивает в процессе своего выполнения.

Для понимания составляющих процесса нужно понимать состояния системы: что программа может считывать или изменять во время своей работы. В любой момент времени нужно понимать какие элементы системы важны для выполнения программы.

Одним из очевидных элементов состояния системы, которые включает в себя процесс - это память. Инструкции располагаются в памяти. Данные, которые программа читает или пишет также располагаются в памяти. Таким образом память, которую процесс может адресовать (так называемое адресное пространство) является частью процесса.

Также частью состояния системы являются регистры. Множество инструкций направлено на то чтобы изменить значение регистров или прочитать их значение и таким образом тоже становятся важной частью работы процесса.

Следует отметить, что состояние машины формируется также из некоторых специальных регистров. Например, IP - instruction pointer - указатель на инструкцию, которую программа исполняет в текущий момент. Еще есть stack pointer и связанный с ним frame pointer, которые используются для управления параметрами функций, локальными переменными и адресами возврата.

Наконец, программы часто обращаются к ПЗУ. Такая информация о I/O (вводе-выводе) должна включать в себя список файлов, открытых процессом в данный момент.

Process API

Для того чтобы улучшить понимания работы процесса изучим примеры системных вызовов, которые должны быть включены в любой интерфейс операционной системы. Эти API в том или ином виде доступны на любой ОС.

- Create: В ОС должен входить какой либо метод, позволяющий создавать новый процессы. Когда вы вводите команду в терминал или запускаете приложение через двойной клик по иконке, посылается обращение к ОС для создания нового процесса и последующего запуска указанной программы.
- Удаление: Раз есть интерфейс для создания процесса, ОС также должна предоставлять возможность для принудительного удаления процесса. Большинство программ естественно будут запущены и завершены сами по себе по

мере их выполнения. В противном случае пользователь хотел бы иметь возможность убить их и таким образом интерфейс для остановки процесса будет не лишним.

- Wait: Иногда полезно дождаться завершения процесса, поэтому обеспечиваются некоторые интерфейсы предоставляющие возможностью ожидания.
- Misc Control: Кроме убийства и ожидания процесса еще существуют другие разнообразные контролирующие методы. Например большинство ОС предоставляют возможность заморозки процесса (остановка его выполнения на некоторый период) и последующее возобновление (продолжение исполнения)
- Status: Существуют различные интерфейсы для получения некоторой информации он статусе процесса, такие как продолжительность его работы или в каком состоянии он сейчас находится.

Создание процесса: детали

Одна из интересных вещей - как же именно программы трансформируются в процессы. Особенно, как ОС поднимает и запускает программу. Как конкретно создается процесс.

В первую очередь ОС должна загрузить код программы и статические данные в память, в адресное пространство процесса. Программы обычно располагаются на диске или твердотельном накопителе в некотором исполняемом формате. Таким образом процесс загрузки программы и статических данных в память требует от ОС возможности прочитать эти байты с диска и расположить их где-то в памяти.

В ранних ОС процесс загрузки выполнялся нетерпеливо (eagerly), т.е. это значит что код загружался в память целиком до того как программа запускалась. Современные ОС делают это лениво (lazily), т.е. Загружая кусочки кода или данных только тогда, когда они требуются программе во время ее выполнения.

После того как когд и статические данные загружены в память ОС нужно выполнить еще несколько вещей перед тем как запустить процесс. Некоторое количество памяти должно быть выделено под стек. Программы используют стек для локальных переменных, параметров функций и адресов возврата. ОС выделяет эту память и отдает ее процессу. Стек также может выделяться с некоторыми аргументами, конкретно она заполняет параметры функции main(), например массивом argc и argv.

Операционная система может также выделять некоторое количество памяти под кучу (heap) программы. Куча используется программами для явно запрашиваемых динамически выделенных данных. Программы запрашивают это пространство, вызывая функцию malloc() и явно очищают, вызывая функцию free(). Куча нужна для таких структур данных как: связанные листы, таблицы хэшей, деревья и др. По началу под кучу выделяется маленькое количество памяти, но со временем в процессе работы программы она может запросить большее количество через библиотечный API вызов malloc(). Операционная система вовлечена в процесс выделения большего количества памяти для того, чтобы помочь в удовлетворении этих вызовов.

Операционная система также будет выполнять задачи инициализации, в частности те, которые относятся к вводу-выводу. Например, в системах UNIX каждый процесс по умолчанию имеет 3 открытых файловых дескриптора, для стандартного потока ввода,вывода,ошибок. Эти дескрипторы позволяют программам считывать ввод из терминала и также выводить информацию на экран.

Таким образом загружая код и статические данные в память, создавая и инициализирую стек, а также выполняя другую работу, относящуюся к выполнению задач ввода-вывода ОС подготавливает площадку для выполнения процесса. В конце концов и остается всего последняя задача: запустить на исполнение программу через ее точку ввода, называемую функцией main (). Переходя к выполнению функции MAIN () ОС передает управление CPU вновь созданному процессу, таким образом программа начинает исполняться.

Состояние процесса

Теперь, когда у нас есть некоторое понимание, что такое процесс и как он создается, перечислим состояния процесса, в которых он может находиться. В самой простой форме процесс может находиться в одном из этих состояний

- Running. В запущенном состоянии процесс выполняется на процессоре. Это значит, что происходит выполнение инструкций.
- Ready. В состоянии готовности процесс готов запуститься, но по каким-то причинам ОС не исполняет его в заданный момент времени.
- Blocked. В заблокированном состоянии процесс выполняет какие-то операции, которые не дают ему быть готовым к исполнению до тех пор, пока не произойдет какое-либо событие. Один из обычных примеров - когда процесс инициализирует операцию IO, он становится заблокированным и таким образом какой-то другой процесс может использовать процессор.

Представить себе эти состояния можно в виде графа. Как мы можем видеть на картинке состояние процесса может меняться между RUNNING and READY на усмотрение ОС. Когда состояние процесса меняется с READY на RUNNING это означает, что процесс был запланирован. В обратную сторону - убран из планировки. В момент, когда процесс становится BLOCKED , например инициализирую операцию IO, ОС ,будет держать его в этом состоянии до наступления некоторого события, например завершение IO. в этот момент переход в состояние READY и возможно моментально в состояние RUNNING если так решит ОС.

Давайте взглянем на пример того, как два процесса проходят через эти состояния. Для начала представим, что оба процесса запущены и каждый использует только CPU. в этом случае, их состояния будут выглядеть следующим образом (РИС). В следующем примере первый процесс через некоторое время работы запрашивает IO и переходит в состояние BLOCKED, предоставляя другому процессу возможность запуска (РИС). ОС видит, что процесс 0 не использует CPU и запускает процесс 1. в о время выполнения процесса 1 - IO завершается и статус процесса 0 меняется на READY. Наконец процесс 1

завершается, а после него процесс 0 запускается, исполняется и заканчивает свою работу.

Структура данных

ОС сама является программой, и также как и любая другая программа имеет некоторые ключевые структуры данных, которые отслеживают разнообразные релевантные куски информации. Для отслеживания состояния каждого процесса в ОС будет поддерживаться некоторый process list для всех процессов в состоянии READY и некоторую доп информацию для отслеживания процессов, которые выполняются в текущий момент. Также ОС должна отслеживать и заблокированные процессы. После завершения IO, также должна пробудить нужный процесс и перевести его в состояние готовности к запуску.

Так например ОС должна сохранить состояние регистров процессора. В момент остановки процесса состояние регистров сохраняется в адресном пространстве процесса, а в момент продолжения его работы - восстановить значения регистров и таким образом продолжить выполнения этого процесса.

Кроме состояний ready, blocked,running существуют еще некоторые другие состояния. Иногда в момент создания процесс может иметь состояние INIT. Наконец процесс может быть помещен состояние FINAL, когда он уже завершился, но информация о нем еще не вычищена. В UNIX системах такое состояние называется процесс-зомби. Это состояние полезно для случаев когда родительский процесс хочет узнать код возврата потомка, например обычно 0 сигнализирует об успешном завершении, а 1 об ошибочном, однако программисты могут делать дополнительные коды вывода, сигнализируя о разных проблемах. При завершении процесс-родитель делает последний системный вызов, например wait() чтобы дождаться завершения работы процесса-потомка и просигнализировать ОС что можно очистить любые данные, связанные с завершенным процессом.

Ключевые моменты лекции:

- Процесс - главная абстракция работающей программы в ОС. В любой момент времени процесс может быть описан по его состоянию: содержимое памяти в его адресном пространстве, содержимое регистров процессора, включая instruction pointer и stack pointer также информацией о IO, например открытыми файлами, которые считываются или записываются.
- Process API состоит из вызовов, которые программы могут делать в отношении процессов. Обычно это вызовы создания, удаления или другие.
- Процесс находится в одном из множества состояний, включая running, ready, blocked. Различные события такие как планировка, исключение из планировки или ожидания могут переводить состояние процесса из одного в другое.
- Process list содержит информацию обо всех процессах в системе. Каждая запись в ней называется process control block, которая в реальности является структурой, которая содержит всю необходимую информацию о конкретном процессе.

Process API

Рассмотрим пример создания процесса в UNIX системе. Он происходит через два системных вызова fork() и exec().

Вызов fork()

Рассмотрим программу, которая выполняет вызов fork(). Результат ее выполнения будет следующим. В первую очередь мы входим в функцию main() и выполняем вывод строки на экран. Строка содержит идентификатор процесса или PID. этот идентификатор применяется в UNIX для того чтобы обращаться к процессу. Следом за ней будет выполнен вызов fork(). В этот момент создается практически точная копия процесса. Для ОС это выглядит так, что в системе запущены как-будто бы 2 копии одной и той же программы, которые обе в свою очередь выйдут из выполнения функции fork(). Вновь созданные процесс-потомок (по отношению к создавшему его процессу-родителю) уже не будет выполняться начиная с функции main(). Следует помнить, что процесс-потомок не является точной копией процесса-родителя, в частности у него есть собственное адресное пространство, собственные регистры, свой указатель на исполняемые инструкции итп. Таким образом, значение, возвращаемое вызывателю функции fork() будет разным. В частности процесс-родитель получит в качестве возврата значение PID процесса ребенка, а ребенок получит значение равное 0. По этим кодам возврата в дальнейшем уже можно разделять процессы и заставлять каждый из них выполнять свою работу. При этом выполнение данной программы не определено строго. После разделения на 2 процесса ОС начинает следить за ними, а следовательно и планировать их работу. В случае выполнения на одноядерном процессоре работу продолжит один из процессов, в данном случае - родительский, а затем управление получит процесс-потомок. При повторном запуске ситуация может сложиться иначе.

Вызов wait()

Рассмотрим следующую программу. В этой программе за счет наличия вызова wait() процесс-родитель всегда будет дожидаться завершения работы процесса-потомка. В этом случае мы получим строго определенный вывод текста на экран.

Вызов exec()

Рассмотрим вызов exec(). Этот системный вызов полезен, когда мы хотим запустить совершенно другую программу. Здесь мы будем вызывать execvp() для запуска программы wc, которая является программой подсчета слов. ЧТо же происходит при вызове exec()? Этому вызову передаются в качестве аргументов имя исполняемого файла и некоторые параметры. После чего происходит загрузка кода и статических данных из этого исполняемого файла и перезатирание собственного сегмента с кодом. Остальные

участки памяти такие как стек и куча переинициализируются. После чего ОС просто исполняет программу, передавая ей набор аргументов. Таким образом мы не создавали новый процесс, мы просто трансформировали текущую запущенную программу в другую запущенную программу. После выполнения вызова exec() в потомке наступает впечатление, что изначальная программа как-будто бы в принципе не запускалась.

Такое усложнение запуска абсолютно нормально для shell оболочки unix, и позволяет этой оболочке исполнять код после вызова fork() но до вызова exec(). Примером такого кода может быть подстройка окружения оболочки под нужды запускаемой программы перед ее непосредственным запуском.

Shell - всего лишь пользовательская программа. Она показывает вам строку приглашения и ждет пока вы в него что-нибудь напишете. В большинстве случа если написать туда имя программы, shell найдет его месторасположение, вызовет метод fork(), а затем чтобы создать новый процесс вызовет какой-либо из типов exec() и дождется его выполнения с помощью вызова wait(). Когда процесс-потомок завершится shell вернется из вызова wait() и выведет снова строку приглашения и будет ждать ввода следующей команды.

Разделение fork() & exec() позволяет shell делать следующие вещи, например: wc file > new_file.

В этом примере вывод программы wc перенаправлен в файл. Способ, которым shell достигает этого достаточно просто - при создании процесса-ребенка перед вызовом exec() shell закрывает стандартный поток вывода и открывает файл new_file, таким образом весь вывод из далее запущенной программы wc будет перенаправлен в файл вместо экрана.

Unix pipe реализованы похожим образом, с разницей что они используют вызов pipe(). В этом случае поток вывода процесса будет подключен к очереди pipe, расположенной в ядре к которой же будет присоединен поток ввода другого процесса.